

SSH Honeypot & SOC Monitoring Platform

Author: V. Gedeon

Project: Cowrie-based SSH Honeypot & SOC

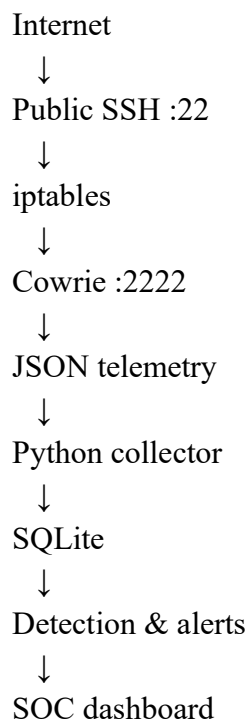
Repository: github.com/vgedeon01/honeypot-project

1. Executive Summary

This project is a defensive security monitoring platform built around an intentionally exposed SSH honeypot.

I deployed **Cowrie** on an Ubuntu VPS, redirected public SSH traffic from port 22 into the honeypot, collected the resulting telemetry, stored it in SQLite, classified attacker commands, generated alerts, and visualized the results through a local SOC dashboard.

The final system provides an end-to-end pipeline:



The project was tested manually and then exposed to unsolicited Internet traffic. The honeypot subsequently observed automated SSH authentication attempts, credential testing, reconnaissance, and post-authentication system fingerprinting.

The project demonstrates practical experience with:

- Linux administration
- SSH security
- Firewall Configuration
- Network configuration
- Honeypot deployment
- Python development
- Log processing
- SQLite
- Detection engineering
- SOC monitoring
- Incident investigation
- Systemd services
- Git/GitHub

2. Project Objectives

The project was built to answer a simple question:

Can I build and operate a small SOC pipeline that detects and analyzes real SSH attacks?

The main objectives were:

1. Deploy an SSH honeypot safely.
2. Keep the real VPS administration interface separated from the honeypot.
3. Collect structured security telemetry.
4. Detect suspicious commands automatically.
5. Store events in a searchable database.
6. Build a simple SOC dashboard.
7. Validate the detections with controlled attacks.
8. Analyze unsolicited attacks from the public Internet.

The focus was on building a working system rather than simply installing an existing honeypot.

3. Security Background

SSH is one of the most frequently exposed administrative services on Internet-facing Linux systems. Public SSH services are regularly scanned and targeted by automated systems attempting common usernames, passwords, and post-authentication commands.

A honeypot provides a controlled environment where this behaviour can be observed without giving attackers access to the real operating system.

Cowrie was selected because it provides an interactive SSH environment and produces structured telemetry about connections, authentication attempts, commands, and sessions.

The project adds a custom monitoring layer around Cowrie to turn this raw telemetry into security events and alerts.

4. System Architecture

The system consists of five main components.

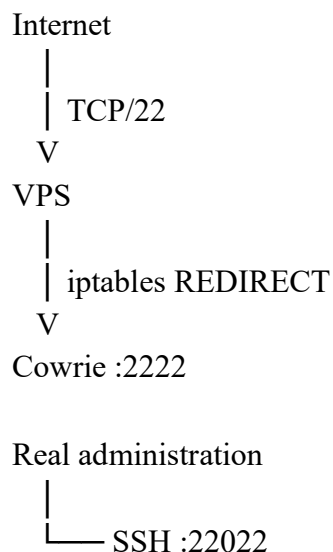
Cowrie

Cowrie provides the SSH honeypot and emulated Linux shell.

Network layer

Public TCP/22 is redirected to Cowrie on TCP/2222.

The real SSH service operates separately on TCP/22022.



Collector

collector.py monitors Cowrie's JSON log and converts events into structured database records.

Database

SQLite stores:

events
sessions
commands
downloads
alerts

SOC dashboard

A Flask dashboard displays live statistics including:

- sessions
- commands
- alerts
- alert severity
- command categories
- source IPs
- top commands

The dashboard listens only on:

127.0.0.1:5000

It is therefore not exposed to the Internet.

5. Network & Security Configuration

Security separation was an important part of the deployment.

Service	Port	Purpose
Public SSH	22	Cowrie honeypot
Cowrie	2222	Internal honeypot listener
Real SSH	22022	Administration
Dashboard	5000	Local monitoring

The public SSH port is redirected with iptables, while the real SSH service was moved away from port 22 and configured for key-based authentication.

The administrative port is restricted through the VPS firewall.

The dashboard is bound to localhost and therefore cannot be accessed directly by external attackers.

This gives the system a clear separation:

Attack traffic → Cowrie

Administrator → Real SSH

SOC monitoring → Local dashboard

6. Telemetry Collection

Cowrie writes events to:

`/opt/cowrie/var/log/cowrie/cowrie.json`

The custom collector continuously processes new events.

It records:

- connection information
- source IP
- session ID
- authentication attempts
- usernames/passwords
- commands
- session duration
- downloads
- alerts

The collector also performs event deduplication so that repeated processing does not create duplicate records.

Commands are classified into behavioural categories:

Category	Examples
Recon	whoami, id, uname, ls

Download	wget, curl
Execution	bash, python, chmod
Persistence	crontab, authorized_keys
Credential access	/etc/passwd, /etc/shadow
Networking	nc, ping, dig
Destructive	rm, mkfs, dd

The database then becomes the central source for both analysis and visualization.

7. Detection Engineering

The detection engine uses transparent command-based rules.

Severity is assigned according to behaviour:

Severity	Example
Low	Basic reconnaissance
Medium	Network discovery
High	Credential access, downloads, persistence, destructive commands

For example:

```
whoami
↓
recon
↓
low alert
```

while:

```
cat /etc/shadow
↓
credential_access
↓
high alert
```

The detection rules were tested independently before deployment.

Detection test result: 21/21 tests passed.

This provided a basic regression test for the command classifier.

The system intentionally focuses on **observable behaviour**, rather than attempting to identify the person behind an IP address.

8. Deployment & Validation

The project was deployed as three systemd services:

```
cowrie.service  
cowrie-soc-collector.service  
cowrie-soc-dashboard.service
```

All three components were tested independently and then together.

Controlled test

An external test connection was made to the public port 22.

Commands included:

```
whoami  
ls  
cat /etc/shadow  
exit
```

The expected results were observed.

whoami produced a reconnaissance alert.

cat /etc/shadow produced a high-severity credential-access alert.

This verified the complete pipeline:

SSH connection



Cowrie



JSON log



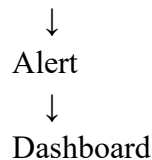
Collector



SQLite



Detection



The test confirmed that the public attack path was reaching Cowrie rather than the real SSH service.

9. Real-World Attack Results

After deployment, the honeypot began receiving unsolicited SSH activity from the public Internet. The anonymized investigation dataset contains **135 unique source addresses**, with activity ranging from simple connection attempts to multi-stage post-authentication behaviour.

The investigation was performed using the collected session, authentication, command, alert, client fingerprint, and file-download telemetry.

The main behaviours observed were:

- automated credential testing
- repeated successful honeypot authentication
- host fingerprinting
- system and process enumeration
- SSH persistence attempts
- credential modification
- cron inspection
- destructive cleanup
- payload/file staging

The important distinction is that these are **observed behaviours inside the honeypot**. A successful Cowrie login does not represent compromise of the underlying VPS.

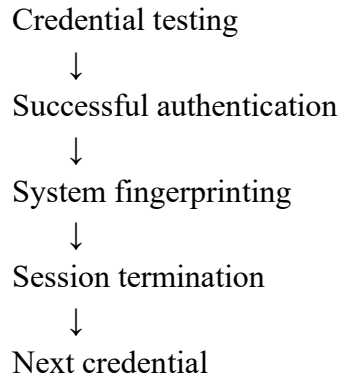
9.1 From Scanning to Post-Authentication Activity

Most observed activity was automated and relatively shallow.

One campaign generated hundreds of short sessions. After successful authentication, the client repeatedly executed a single system-fingerprinting command:

```
uname -s -v -n -r -m
```

The same pattern occurred across many sessions, indicating a workflow of:



This type of activity is useful as a baseline because it represents common automated SSH behaviour.

However, several other campaigns progressed considerably further.

9.2 The Deepest Observed Campaign

The most significant investigation involved **Attacker 102** in the anonymized dataset.

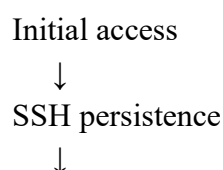
The source generated:

- **68 events**
- **3 sessions**
- **18 commands**
- **8 alerts**
- activity over approximately two minutes

The first successful session lasted approximately 60 seconds and began with a root login.

The attacker immediately attempted to establish SSH persistence by manipulating `.ssh/authorized_keys`. This was followed by host reconnaissance, password modification, process/system inspection, cron inspection, and destructive cleanup.

The sequence was significantly more involved than the automated fingerprinting campaigns:



Credential modification
↓
Host reconnaissance
↓
Process / resource discovery
↓
Cron inspection
↓
Cleanup / destructive activity

The detection engine identified:

- **2 persistence behaviours**
- **1 destructive behaviour**
- multiple reconnaissance behaviours

The investigation therefore moved from simple “SSH brute force” into a genuine **post-authentication attack workflow**.

9.3 Repeated Persistence Campaign

The investigation also revealed that the persistence behaviour was not isolated.

Several sources used the same distinctive SSH client fingerprint and executed substantially identical commands to modify `.ssh/authorized_keys`.

The repeated workflow was:

Successful root login
↓
Modify SSH configuration/files
↓
Replace `authorized_keys`
↓
Install attacker-controlled public key
↓
Attempt to preserve future access

The same public-key persistence pattern appeared across multiple sources and sessions.

From a SOC perspective, this is significant because it provides a possible **campaign-level correlation**.

The individual source addresses are different, but the combination of:

- SSH client fingerprint
- command sequence
- persistence technique
- attacker-controlled key material
- timing and session behaviour

creates a stronger relationship than IP address alone.

This is the type of correlation that could be developed into a future detection rule.

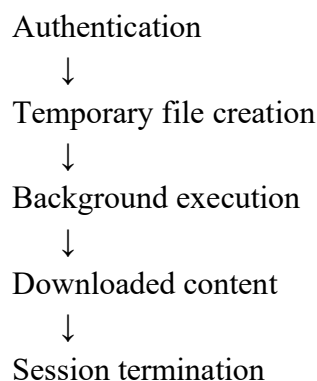
9.4 Payload Staging

A separate source showed another interesting behaviour.

After one failed authentication attempt, the same client successfully authenticated and immediately executed a command that created a temporary file in /tmp and placed the process into the background.

Cowrie then recorded a file-download event and stored the resulting content with a SHA-256 hash.

The sequence was approximately:



The extremely short session — less than one second after authentication — suggests the interaction was automated rather than manually operated.

This is a particularly useful example because it shows why **file-download telemetry matters**. A command that appears relatively simple becomes much more significant when correlated with the subsequent file event.

9.5 Overall Assessment

The observed Internet activity can therefore be divided into several behavioural levels:

Level 1 — Scanning

Connection / authentication attempts

Level 2 — Fingerprinting

uname / hostname / system identification

Level 3 — Enumeration

Processes / memory / filesystem / environment

Level 4 — Persistence

SSH key modification / cron inspection

Level 5 — Payload activity

File staging / downloads / cleanup

The majority of observed sessions remained at the first two levels. A smaller number progressed into deeper post-authentication behaviour.

That difference is one of the most valuable findings from the deployment: **not every SSH attack behaves the same way, and session-level telemetry allows the analyst to distinguish automated scanning from more involved attack workflows.**

10. SOC Investigation Case Studies

The following investigations are presented using a SOC-style approach: **What happened, when it happened, who/what was involved, where it occurred, why the behaviour is significant, and what evidence supports the assessment.**

Because the public report is anonymized, sources are identified only by their investigation IDs.

10.1 Case Study 1 — Deep Post-Authentication Attack

Executive Assessment

Severity: High

Confidence: High

Primary behaviours: Persistence, credential modification, reconnaissance, destructive activity

This was the deepest and most security-relevant interaction observed in the dataset.

What happened?

The attacker successfully authenticated to the Cowrie SSH honeypot and immediately attempted to establish persistent access through SSH.

The session then progressed into host reconnaissance and system manipulation.

Observed behaviours included:

- modification of .ssh/authorized_keys
- attacker-controlled SSH key installation
- root password modification
- CPU and memory inspection
- process inspection
- environment/user discovery
- cron inspection
- cleanup/destructive commands

The detection engine generated high-severity alerts for persistence and destructive behaviour.

When?

The campaign was observed on **27 September 2026**, beginning at approximately **14:33 UTC**.

The primary session lasted approximately one minute, with additional connections shortly afterwards.

Who?

The source is anonymized as **Attacker 102**.

No attempt is made to attribute the activity to a specific individual or organization.

The available evidence identifies an automated SSH client and its behaviour, not the human operator behind it.

Where?

The activity occurred against the project's **public SSH honeypot**, not the real administrative SSH service.

This distinction is important:

Public TCP/22



iptables



Cowrie



Attacker interaction

The underlying VPS administration interface remained separate.

Why?

The exact motivation cannot be established from the telemetry.

However, the sequence provides evidence for a likely operational objective: **maintain access after the initial authentication**.

Installing an SSH public key into `authorized_keys` is particularly significant because it can provide future authentication without relying on the original password.

The subsequent system enumeration suggests the attacker was also attempting to understand the environment and its resources.

The cron inspection may indicate interest in persistence mechanisms, scheduled execution, or existing attacker activity.

These are behavioural assessments, not claims about the operator's identity or intent.

How?

The attack progressed in stages:

1. Successful SSH authentication



2. Attempt SSH persistence



3. Modify authentication state



4. Inspect host resources
- ↓
5. Inspect processes/environment
- ↓
6. Inspect cron
- ↓
7. Perform cleanup/destructive actions

SOC Interpretation

This is materially different from ordinary SSH scanning.

The attacker demonstrated **post-authentication objectives** rather than simply checking whether credentials worked.

The strongest indicator was the attempt to install an attacker-controlled SSH key. That changes the analytical question from:

“Did someone log in?”

to:

“Did the actor attempt to preserve access?”

The answer, based on the honeypot telemetry, is **yes**.

10.2 Case Study 2 — Repeated SSH Persistence Campaign

Executive Assessment

Severity: High

Confidence: High

Primary behaviour: SSH persistence

Correlation: Strong behavioural similarity across sources

Several anonymized sources performed almost identical persistence actions.

The first stage was a successful SSH login, frequently using a root account.

The attacker then attempted to manipulate .ssh and replace authorized_keys with an attacker-controlled key.

Why is this interesting?

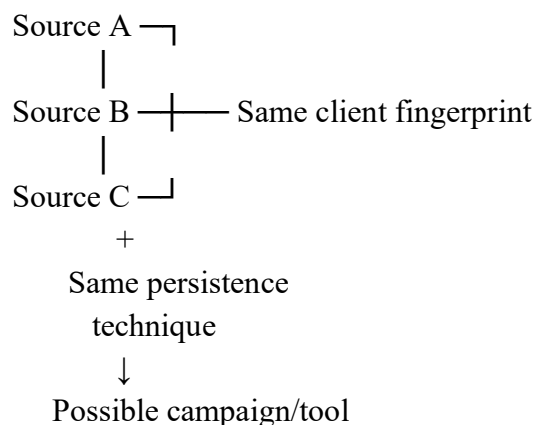
The activity was not observed only once.

Multiple sources showed:

- the same SSH client fingerprint
- the same persistence command structure
- the same attacker-controlled key
- the same general sequence
- extremely short sessions

For example, the anonymized dataset records the same client fingerprint across several of these sessions.

This creates a useful SOC correlation:



Likely objective

The most plausible behavioural explanation is **automated persistence establishment**.

The attacker appears to be attempting to make future SSH access possible through a key rather than relying solely on the credentials used for the initial login.

The evidence supports the behaviour, but not attribution to a specific actor.

Detection opportunity

This campaign suggests a future detection rule that combines:

Successful SSH login
+
authorized_keys modification
+
known client fingerprint
+
repeated appearance

Such a rule would be more useful than detecting each event independently.

10.3 Case Study 3 — Rapid Payload Staging

Executive Assessment

Severity: Medium–High

Confidence: High

Primary behaviours: Authentication, temporary file staging, background execution, file download

This case was much shorter but technically interesting.

The client first attempted authentication unsuccessfully and then immediately established a successful session.

Within roughly one second it executed a command that:

- wrote data to a temporary file
- changed into /tmp
- launched a background process
- caused Cowrie to record a downloaded file

Cowrie captured the resulting file and generated a SHA-256 hash.

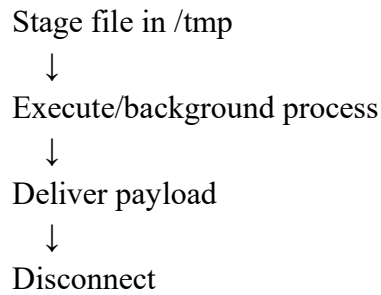
Why?

The exact purpose of the payload cannot be established without safely analyzing the captured file.

However, the behaviour is consistent with a common automated pattern:

Gain access





The extremely short session duration is another indicator that this was automated.

SOC response

In a real environment, this sequence would justify immediate investigation of:

1. the downloaded file hash
2. the file contents in an isolated sandbox
3. the source/session timeline
4. the SSH client fingerprint
5. related sessions using the same fingerprint
6. other hosts showing the same hash

The honeypot prevented this activity from reaching the real operating system while still providing the telemetry necessary for investigation.

10.4 Case Study 4 — Automated Fingerprinting

A fourth case is useful as a baseline.

One source generated a very large number of short SSH sessions. After successful authentication, each session executed the same uname fingerprinting command and then ended.

This behaviour was much less advanced than the previous cases.

Likely objective

The most reasonable interpretation is automated **host fingerprinting**.

The attacker appears to be determining what operating system/kernel environment is present before deciding whether further activity is worthwhile.

Why include it?

Because a SOC analyst should not treat every successful login as a major incident.

The difference between this activity and the deep campaign is substantial:

Behaviour	Fingerprinting	Deep campaign
Authentication	Yes	Yes
Host discovery	Basic	Extensive
Persistence	No	Yes
Credential modification	No	Yes
Destructive activity	No	Yes
Payload activity	No	Observed
Investigation priority	Lower	Higher

This demonstrates the importance of **severity based on behaviour rather than authentication success alone**.

10.5 Investigation Summary

The cases show a progression in attacker behaviour:

Automated scanning



Credential testing



Successful authentication



System fingerprinting



Host enumeration



Persistence



Payload staging



Cleanup / destructive activity

The majority of observed traffic stopped early in this chain.

The most valuable SOC finding was the smaller set of sessions that progressed beyond authentication into persistence and system manipulation.

The project was able to identify those differences because it correlated **authentication, session, command, client fingerprint, file, and alert telemetry** rather than treating each log event independently.

This is the main investigative value of the project: converting thousands of low-level honeypot events into a small number of understandable attack narratives.

11. Limitations & Lessons Learned

The project also exposed several practical limitations.

Honeypot $\neq$ real compromise

A successful honeypot login means that Cowrie accepted the credentials. It does not mean the real VPS was compromised.

IP $\neq$ attacker identity

An IP address identifies a network endpoint, not necessarily the person operating it. Cloud infrastructure, compromised machines, proxies, and shared systems can all affect attribution.

Pattern-based detection has limits

The current classifier relies on command patterns. Obfuscation or previously unknown commands may bypass the existing rules.

Internet data is noisy

Some connections contain only authentication attempts. Others perform reconnaissance. Some terminate immediately.

This makes correlation between sessions, commands, and alerts important.

Testing must be separated from real traffic

The author's own testing generated valid honeypot events. These were excluded from attacker statistics to avoid contaminating the dataset.

The main lesson was that a useful security monitoring system needs both **good telemetry and careful interpretation**.

12. Future Improvements

The current system provides a strong foundation for further development.

Potential improvements include:

- behavioural sequence detection
- command obfuscation detection
- automated IOC extraction
- richer dashboard visualizations
- session timelines
- malware/download analysis
- longer-term attack statistics
- automated detection testing
- stronger outbound network restrictions
- integration with external threat-intelligence sources

A future detection could identify sequences such as:

Login
↓
Recon
↓
Download
↓
Execute
↓
Persistence

rather than evaluating every command independently.

13. Conclusion

This project demonstrates an end-to-end defensive security workflow.

I designed and deployed an SSH honeypot, separated it from the real administration interface, built a custom telemetry collector, created a SQLite security database, implemented command detection, generated alerts, and developed a local SOC dashboard.

The system was validated through controlled attacks and then exposed to real Internet traffic. The collected data demonstrated automated credential testing and post-authentication reconnaissance against the public SSH service.

The project combines practical experience in:

**Linux • SSH • Networking • Python • SQLite • Detection Engineering • SOC Monitoring
• Honeypots • Incident Analysis • Git**

The main outcome is not simply a working honeypot, but a complete pipeline for turning unsolicited network activity into structured security data that can be investigated and presented.